SethuCMS

One CMS for every database

Design Documentation

October 2026

Abstract

SethuCMS is a database-agnostic CMS and admin layer. It connects to SQL, document, key-value and realtime databases, including cloud-hosted ones such as Firebase, Supabase, Neon, MongoDB Atlas, AWS, Google Cloud and Azure, and gives every client stack one REST/-GraphQL API and one admin UI.

Status (0.1.0, released October 2026): the core, six adapters (memory, PostgreSQL, SQLite, MySQL/MariaDB, MongoDB, Firestore), the gateway, the admin app, drafts, publish, live preview, single sign-on, the change feed and webhooks are built, and all ten packages are on npm under `@sethucms`. PostgreSQL, MySQL, MariaDB and SQLite are tested against real servers; MongoDB and Firestore only against stand-ins. A hosted gateway with the admin app runs on Vercel with its state in Firestore. The early sections describe the full design, parts of which are still future work; the later sections describe what is built and how to build, run, deploy and use it. The Roadmap section lists what is done and what is next. Time and file-count figures are planning estimates, not benchmarks.

Contents

1 Overview **7**

1.1 Problem . . . 7
1.2 Product . . . 7
1.3 Users . . . 7
1.4 Scope . . . 7
1.5 Principles . . . 8

2 What to Build **8**

2.1 Components . . . 8
2.2 Adapter families . . . 9
2.3 Per-adapter files . . . 9
2.4 Dependency rules (enforced in CI) . . . 9
2.5 Definition of done for an adapter . . . 9

3 Language Decision **9**
3.1 Recommendation . . . 9
3.2 Why TypeScript first . . . 10
3.3 Where TypeScript is weaker . . . 10
3.4 Options compared . . . 10
3.5 Language freedom for adapters . . . 11

4 Architecture **11**
4.1 System context . . . 11
4.2 Layers . . . 11
4.3 Read request flow . . . 11
4.4 Connect-and-introspect flow . . . 12
4.5 Capability handling . . . 12
4.6 Multi-tenant bring-your-own-database . . . 12
4.7 Deployment view . . . 12

5 Adapter Contract **12**
5.1 Required operations . . . 13
5.2 Capability flags . . . 13
5.3 Mapping rules per family . . . 13
5.4 Error model . . . 13
5.5 Conformance suite . . . 14
5.6 Adding a new adapter . . . 14

6 Data Model and Schemas **14**
6.1 Control-plane entities . . . 14
6.2 Canonical content model . . . 14
6.3 Type mapping (canonical to native) . . . 15
6.4 Portable query AST . . . 15
6.5 Rules . . . 16
6.6 Control-plane DDL . . . 16
6.7 Query AST JSON Schema . . . 18

7 Security **19**
7.1 Threat model . . . 20
7.2 Credentials . . . 20
7.3 Least privilege . . . 20
7.4 Network controls . . . 20

7.5 Injection defence . . . 21
7.6 Access control, audit and testing . . . 21

8 API **21**
8.1 REST endpoints (v1) . . . 21
8.2 Counting records . . . 22
8.3 Singleton collections . . . 22
8.4 Pagination, errors and versioning . . . 22

9 SDKs **23**
9.1 Hand-written layer . . . 23
9.2 TypeScript SDK 0.2.0 . . . 23
9.3 Rules . . . 24

10 Deployment **24**
10.1 Environments and services . . . 24
10.2 Cloud identity and networking . . . 24
10.3 Configuration, observability, CI/CD . . . 24

11 Gateway **25**
11.1 Many databases at once . . . 25
11.2 Security controls . . . 26
11.3 Browser access (CORS) . . . 26
11.4 Counts and scheduled publishing . . . 26
11.5 Known limits . . . 26

12 Drafts, Publishing and Live Preview **26**
12.1 What the editor sees . . . 27
12.2 Three protections . . . 27
12.3 Scheduled publishing . . . 27
12.4 Storage . . . 28
12.5 Live preview . . . 28
12.6 API summary . . . 28

13 Build and Run **28**
13.1 Requirements . . . 28
13.2 Build and start . . . 29
13.3 Connect a database . . . 29
13.4 Tests . . . 29

13.5 Production settings 29

14 CI/CD Pipelines **29**

14.1 What runs where 30
14.2 How the linked repositories are built 30
14.3 Release image 31
14.4 Not covered yet 31
14.5 Tooling decision: Node and pnpm 31

15 Deploy to Production **31**

15.1 What you deploy 31
15.2 Steps 31
15.3 Day-two operations 32
15.4 Limits to plan around 32
15.5 Checklist for an agent doing a deploy 32

16 Logging and Monitoring **32**

16.1 What is logged 33
16.2 Metrics 33
16.3 The audit log 33
16.4 Limits 33

17 Moving Data Between Databases **34**

17.1 How it works 34
17.2 What the plan blocks 34
17.3 Ids and references 34
17.4 What it does not do 34
17.5 Verified 35

18 Database adapters **35**

18.1 Shared rules 35
18.2 SQLite 35
18.3 MySQL and MariaDB 36
18.4 MongoDB 36
18.5 Firestore 37
18.6 Running the live tests against your own server 37
18.7 Writing your own adapter 37

19 Open-source release checklist **38**

19.1 Before making anything public . . . 38
19.2 The organisation profile . . . 38
19.3 Day of release . . . 38
19.4 After release . . . 39

20 Sponsorship and funding **39**
20.1 Where the money can go . . . 39
20.2 Setting it up (you do these steps) . . . 39
20.3 What the money pays for . . . 40
20.4 Rules for sponsors . . . 40
20.5 Before you ask for money . . . 40
20.6 Open questions for you . . . 41

21 Single sign-on (OpenID Connect) **41**
21.1 How it works . . . 41
21.2 Set it up . . . 41
21.3 Limits . . . 42

22 Several gateways, and cloud deploys **42**
22.1 Shared state . . . 42
22.2 The worker . . . 42
22.3 Deploy workflows . . . 42

23 Adapters in other languages (gRPC) **43**

24 Change feed and webhooks **44**
24.1 Reading the feed . . . 44
24.2 Webhooks (admin only, API only) . . . 44
24.3 Not built . . . 44

25 Use it from your app **44**
25.1 Install the SDK . . . 45
25.2 Where the token lives . . . 45
25.3 Example apps . . . 45
25.4 From install to a live site . . . 45
25.5 What is not there yet . . . 46

26 Hosted gateway, admin app and your first content connection **46**
26.1 What is running . . . 46

26.2 Sign in to the admin app . . . 46
26.3 Add your content database . . . 46
26.3.1 The mistake to avoid . . . 47
26.4 Read the content from a website . . . 47
26.5 The state store in serverless mode . . . 48
26.6 Operating notes . . . 48

27 Roadmap **48**
27.1 Hand-written files for the MVP (estimate) . . . 50
27.2 Risks . . . 50

A Adapter Protocol (proto) **50**

B Architecture Decision Records **52**

1 Overview

1.1 Problem

Content tools are tied to one database. Strapi and Directus are SQL-only, FireCMS is Firestore-first, and internal-tool builders such as Retool and Appsmith reach many databases but are not content CMSs. Teams running several stacks end up with several admin panels.

1.2 Product

SethuCMS is a database-agnostic CMS and admin layer:

- Connect any supported database, including bring-your-own cloud databases.
- Introspect it, describe it as a canonical content model, and edit it through one admin UI.
- Expose one REST/GraphQL API so every client stack uses the same contract.
- Keep credentials server-side, read-only by default, with a full audit trail.

1.3 Users

User	Needs
Developer	Connect a database, get an API and SDK
Editor	Edit content without touching the database
Admin / security	Control who sees what, rotate credentials, read audit logs
Tenant (multi-tenant mode)	Bring their own database safely

1.4 Scope

In scope for the MVP:

- Adapters: Postgres, MySQL, MongoDB, Firestore, SQL Server, DynamoDB.
- Introspection, content-model overlay, CRUD, cursor pagination.
- RBAC with field and row rules, SSO/OIDC.
- Encrypted credential vault, audit log, SSRF guard.
- REST, GraphQL and OpenAPI; a TypeScript SDK and generated Dart and Python SDKs.
- Admin UI.

Later: Cosmos DB, Spanner, BigQuery, Redshift, Redis, Firebase RTDB, realtime subscriptions across all adapters, gRPC, and adapters in other languages through the adapter protocol.

Non-goals:

- Not a database or migration tool. It reads and writes the customer's database; it does not replace it.
- Not a cross-database transaction manager. Cross-source relations are resolved at the gateway, not made atomic.
- Not a BI or analytics tool. Warehouse adapters (BigQuery, Redshift) are read-only by default.

1.5 Principles

1. **Security first.** Credentials never reach a browser or mobile app.
2. **Honest capabilities.** Each adapter declares what it can and cannot do; the UI follows it.
3. **Adapters are isolated.** One slow or hostile database cannot starve the rest.
4. **One contract.** Clients never talk to a database directly.

2 What to Build

2.1 Components

Component	Purpose	MVP
`packages/core`	Canonical content model, query AST, capability flags, errors	Yes
`protocol/`	Language-neutral adapter contract (proto and OpenAPI)	Yes
`packages/adapter-sdk`	Base adapter, registry, conformance test kit	Yes
`packages/adapters/*`	One adapter per database	6
`packages/vault`	Envelope encryption, KMS, key rotation, cloud-identity tokens	Yes
`packages/auth`	OIDC, RBAC/ABAC, field and row rules	Yes
`packages/audit`	Append-only audit writer with redaction	Yes
`packages/network-guard`	SSRF blocklist, TLS rules, egress and tunnels	Yes
`apps/api`	Gateway: REST, GraphQL, OpenAPI	Yes
`apps/worker`	Isolated adapter runners, introspection jobs, real-time	Yes
`apps/admin`	Editor and admin UI	Yes
`db/control-plane`	Migrations for tenants, connections, metadata, permissions, audit	Yes
`sdk-ts`	Hand-tuned TypeScript client	Yes
`sdks/dart`, `sdks/python`	Generated from OpenAPI plus a small hand-written layer	Yes
`infra/`	Docker, Terraform, Kubernetes	Yes
`tests/`	Conformance, security, end-to-end	Yes

2.2 Adapter families

Family	Adapters	Notes
SQL	postgres, mysql, mssql	postgres covers Supabase, Neon, RDS, Aurora, Cloud SQL, AlloyDB, Azure PostgreSQL
Document	mongodb, firestore, cosmosdb	mongodb also covers Atlas, DocumentDB (feature gaps), Cosmos Mongo API
Key-value	dynamodb, redis	Key conditions only; scans are explicit and costly
Realtime	firebase-rtdb	Path-based
Analytics	bigquery, redshift	Read-only by default; later
Other cloud	spanner, athena, opensearch	Later, on demand

2.3 Per-adapter files

Every adapter folder has the same layout, so adding a database is a copy-and-fill job:

```
package.json  tsconfig.json  README.md
src/index.ts  adapter.ts  config.schema.ts  connection.ts  auth.ts
src/introspect.ts  capabilities.ts  query-compiler.ts  type-map.ts
src/operations.ts  pagination.ts  realtime.ts  errors.ts
tests/conformance.test.ts  compiler.test.ts  introspect.test.ts  security.test.ts
```

2.4 Dependency rules (enforced in CI)

- `core` imports nothing internal.
- `adapters/*` depend only on `core` and `adapter-sdk`.
- `admin` talks only to `api` or `sdk-ts`, never to a database.
- Only `api` and `worker` may import `vault`.

2.5 Definition of done for an adapter

1. Passes the shared conformance suite.
2. Passes the security suite (injection, identifier allowlist).
3. Declares accurate capability flags.
4. Documents supported services, auth modes and limits in its README.

3 Language Decision

This is a recommendation based on the project's needs, not a measured benchmark.

3.1 Recommendation

Use **TypeScript for the whole MVP**. Add **Go** later for performance-critical or third-party adapters through the adapter protocol. Dart and Python SDKs are generated from OpenAPI.

Part	Language	Why
core, adapter-sdk, api, worker	TypeScript (Node.js)	One language across server and UI, shared types, strong official SDKs for Firestore, MongoDB, DynamoDB, Postgres
Admin UI	TypeScript, React (Refine)	Backend-agnostic data providers, same types as the API
Control-plane migrations	SQL	Plain and reviewable
SDK: TypeScript	TypeScript	Hand-tuned, first class
SDK: Flutter	Dart	Generated plus a small hand-written layer
SDK: Python	Python	Generated plus a small hand-written layer
Third-party or heavy adapters (later)	Go or any language	Separate process through the protocol

3.2 Why TypeScript first

- The query AST, content model and capability flags are shared by the gateway, adapters, SDK and admin UI. One type system avoids drift.
- Database SDK coverage is best on Node for the targets that matter (Firebase Admin, MongoDB, DynamoDB, `pg`/`mysql2`, Azure and Google clients).
- It is the fastest path to a working MVP for one or two engineers.

3.3 Where TypeScript is weaker

- CPU-heavy work and very high connection counts are better in Go or Rust.
- A single event loop needs care: run adapters in worker processes, not inside the API process.

3.4 Options compared

Option	Strength	Weakness	Verdict
TypeScript everywhere	Shared types, fastest MVP, best DB SDK coverage	Weaker for CPU-bound work	**MVP choice**
Go core	Fast, small binaries, great concurrency	Admin UI still needs TypeScript; two type systems	Good for later adapters
Python core	Rich data libraries	Slower, weaker typing for a gateway	SDK and scripting only
Rust core	Performance and safety	Slow to build, thin cloud-SDK coverage	Not worth it for the MVP
Java / Kotlin	Mature DB drivers	Heavier, slower iteration	Only if a client requires it

3.5 Language freedom for adapters

Adapters run as separate processes speaking the protocol in `adapter.proto`. The gateway does not care what language an adapter is written in. First-party adapters ship in TypeScript and run through the same protocol, with an in-process fast path allowed for trusted ones.

4 Architecture

4.1 System context

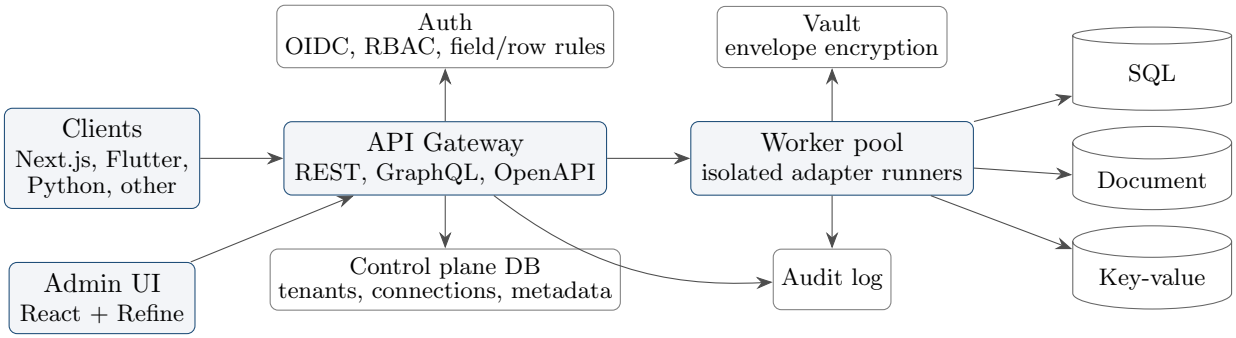


Figure 1: System context. Clients and the admin UI only ever call the gateway.

4.2 Layers

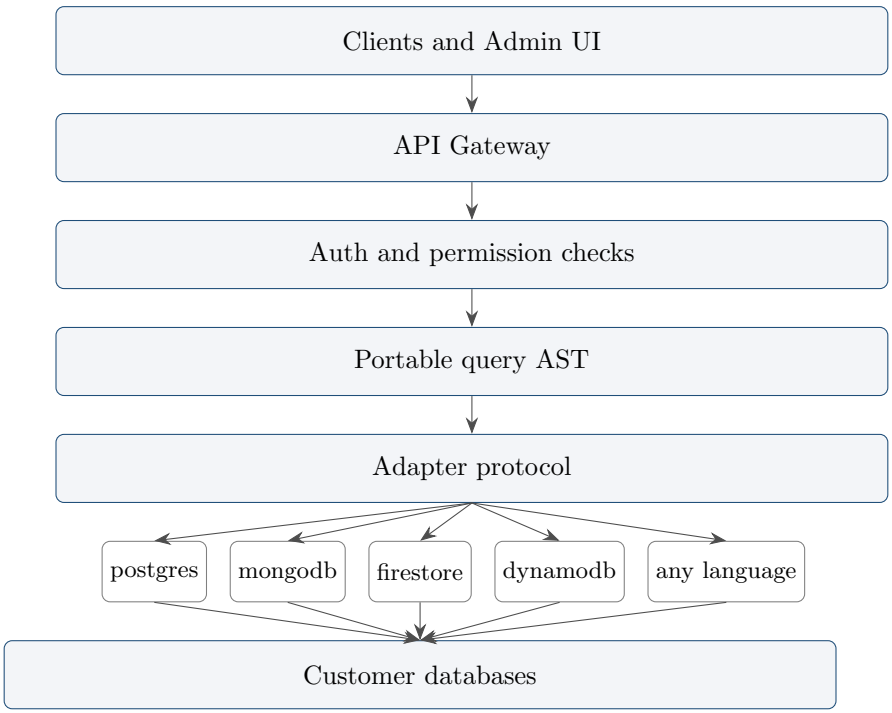


Figure 2: Request path from client to database.

4.3 Read request flow

1. The client calls `GET /v1/connections/{id}/collections/orders/records`.

2. The gateway verifies the token and resolves roles through Auth.
3. The gateway loads connection and collection metadata from the control plane.
4. It builds the query AST and applies field and row rules.
5. It calls `find(ast)` on the worker over the adapter protocol.
6. The worker fetches a short-lived credential or IAM token from the vault.
7. The adapter runs a parameterised native query against the customer database.
8. The worker returns records and a next cursor; the gateway appends an audit event and responds.

4.4 Connect-and-introspect flow

1. An admin posts a new connection (type, host, secret reference).
2. The network guard validates the host (SSRF, TLS, allowlist).
3. The vault encrypts and stores the secret.
4. The control plane saves the connection, read-only by default.
5. A worker reads the catalog or samples documents and saves a draft content model.
6. The admin reviews the draft schema before any collection is exposed.

4.5 Capability handling

When a query uses something an adapter cannot do, the gateway has three outcomes: compile it to the native query (supported), reject it with `CAPABILITY_UNSUPPORTED` (not supported), or emulate it at the gateway and flag it in the response (emulatable). For example, DynamoDB has no arbitrary sort, so sorting by any column is rejected; Firestore has limited OR and inequality filters, so some filters are rejected or emulated and marked as such.

4.6 Multi-tenant bring-your-own-database

The control plane holds tenants, connections, secret references, collection metadata, and roles. The data plane runs one isolated worker per connection (or per tenant). Each connection gets its own pool limits so one tenant's slow query cannot starve another.

4.7 Deployment view

A load balancer with a WAF fronts stateless API pods. API pods use the control-plane Postgres and a job queue. Worker pods read secrets through KMS and reach customer databases through a NAT with static egress IPs, so customers can allowlist the platform. Private networking (PrivateLink, Private Service Connect, Azure Private Link) and SSH tunnels are supported alongside.

5 Adapter Contract

An adapter is the only code that knows a database's native language. It turns the portable query AST into native queries and native results into canonical records.

5.1 Required operations

Operation	Purpose
`connect(config)`	Open a connection or client using resolved credentials
`introspect()`	Return collections, fields, types, keys, relations, indexes
`capabilities()`	Return capability flags
`find(query)`	List records by filter, sort, projection, cursor
`get(collection, id)`	Fetch one record
`create / update / delete`	Write operations (blocked on read-only connections)
`batch(ops)`	Multiple writes, atomic only if `transactions` is supported
`subscribe(query)`	Optional realtime stream
`close()`	Release resources

5.2 Capability flags

```
interface Capabilities {
  joins: boolean;
  transactions: "none" | "single-document" | "multi-document";
  filters: { or: boolean; inequality: "full" | "single-field" | "none";
             fullText: boolean };
  sort: "any-field" | "indexed-only" | "key-only";
  pagination: { cursor: true; offset: boolean };   // cursor is mandatory
  aggregations: boolean;
  realtime: boolean;
  schema: "enforced" | "inferred";
  writes: boolean;
}
```

The gateway and admin UI use these flags to hide or disable unsupported controls instead of doing silent client-side work.

5.3 Mapping rules per family

Concern	SQL	Document	Key-value
Introspection	Read catalog (information_schema)	Sample documents, infer draft schema	Read key schema, sample items
Filters	Parameterised SQL	Native filter objects	Key conditions only
Pagination	Keyset	startAfter cursor	LastEvaluatedKey
Relations	Foreign keys	Reference fields or embedding	Denormalised
Transactions	Full	Limited	Limited

5.4 Error model

Adapters map native errors to a fixed set so clients see one vocabulary: `CONNECTION_FAILED`, `AUTH_FAILED`, `NOT_FOUND`, `CONFLICT`, `VALIDATION_FAILED`, `CAPABILITY_UNSUPPORTED`, `RATE_LIMITED`, `TIMEOUT`, `READ_ONLY`, `INTERNAL`.

5.5 Conformance suite

A shared test kit runs the same scenarios against every adapter:

1. introspect returns stable collections and field types;
2. create, get, update, delete round trip;
3. cursor pagination returns each record exactly once;
4. unsupported filters return `CAPABILITY_UNSUPPORTED`;
5. a read-only connection rejects writes with `READ_ONLY`;
6. hostile identifiers and operators are rejected (injection);
7. errors map to the standard set.

An adapter is not releasable until all pass.

5.6 Adding a new adapter

Copy the adapter template; fill in the config schema, connection, introspection, query compiler and type map; declare capabilities honestly; register it in the registry; pass the conformance and security suites. Nothing in core, api or admin changes.

6 Data Model and Schemas

Two models exist and must not be mixed. The **control plane** is the platform's own Postgres database (tenants, connections, metadata, permissions, audit). The **canonical content model** is the platform's description of a customer's data; it is stored in the control plane, never inside the customer's database.

6.1 Control-plane entities

6.2 Canonical content model

A **collection** has a name, a native name, fields, relations and capabilities. A **field** has a name, a canonical type, *required* and *unique* flags, validation rules and UI hints. A **relation** has a name, a kind and a mapping to a target collection. Canonical types: string, text, integer, decimal, boolean, datetime, date, json, array, reference, binary, geopoint.

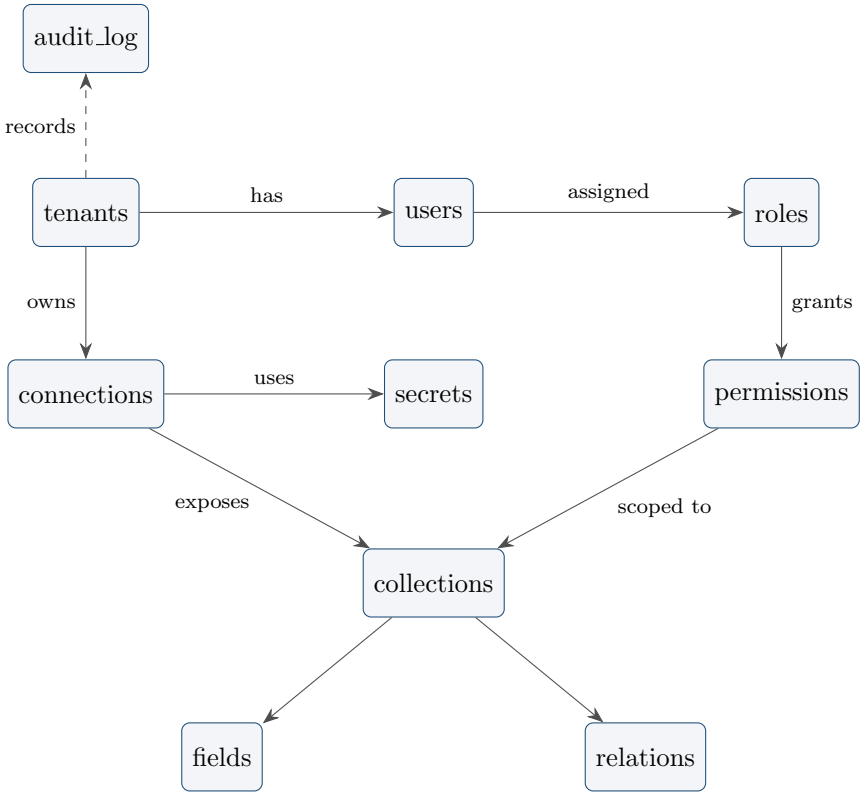


Figure 3: Control-plane entities. The full DDL follows in Section 6.6.

6.3 Type mapping (canonical to native)

Canonical	Postgres	MySQL	MongoDB	Firestore	DynamoDB
string	varchar/text	varchar	string	string	S
integer	int/bigint	int/bigint	int32/int64	integer	N
decimal	numeric	decimal	decimal128	double	N
boolean	boolean	tinyint(1)	bool	boolean	BOOL
datetime	timestamptz	datetime	date	timestamp	S (ISO) or N
json	jsonb	json	object	map	M
array	array or jsonb	json	array	array	L
reference	foreign key	foreign key	ObjectId	document ref	key string
geopoint	point	point	GeoJSON	geopoint	M

Adapters own the exact conversion in `type-map.ts`. Lossy conversions must be flagged during introspection.

6.4 Portable query AST

A query has a collection, an optional filter (`and`/`or`/`not` over comparisons), sort, projection and a page (limit and cursor). Comparison operators: `eq`, `ne`, `lt`, `lte`, `gt`, `gte`, `in`, `nin`, `contains`, `startsWith`, `exists`.

```
{
  "collection": "orders",
  "filter": { "and": [
    { "field": "status", "op": "eq", "value": "paid" },
```

```
    { "field": "total", "op": "gte", "value": 100 }
  ]},
  "sort": [{ "field": "created_at", "dir": "desc" }],
  "projection": ["id", "status", "total", "created_at"],
  "page": { "limit": 50, "cursor": null }
}
```

Field names are always validated against introspected metadata before compilation; they are never interpolated into native queries.

6.5 Rules

1. Metadata lives in the control plane, never in the customer's database.
2. Cursor pagination is the default everywhere.
3. Cross-source relations resolve at the gateway with batching, not inside any database.
4. Validation happens in the CMS layer for schemaless stores.

6.6 Control-plane DDL

```
-- SethuCMS control-plane schema (PostgreSQL)
-- Platform metadata only. Customer data never lives here.

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE tenants (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name        text NOT NULL,
    created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE users (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    email         text NOT NULL,
    oidc_subject  text NOT NULL,
    created_at    timestamptz NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, email),
    UNIQUE (tenant_id, oidc_subject)
);

-- Encrypted connection secrets (envelope encryption; key reference points to KMS).
CREATE TABLE secrets (
    id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    kms_key_ref  text NOT NULL,
    ciphertext   bytea NOT NULL,
    auth_kind    text NOT NULL CHECK (auth_kind IN
                   ('password','service_account','iam_token','assume_role','managed_identity','
    secret_ref')),
    key_version  integer NOT NULL DEFAULT 1,
    created_at   timestamptz NOT NULL DEFAULT now(),
    rotated_at   timestamptz
);

CREATE TABLE connections (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name          text NOT NULL,
```

```
    adapter_type  text NOT NULL,           -- postgres, mysql, mssql, mongodb, firestore,
    dynamodb, ...
    config        jsonb NOT NULL DEFAULT '{}'::jsonb,  -- non-secret settings: host, port, db,
    tls, pool caps
    secret_id     uuid REFERENCES secrets(id),
    mode          text NOT NULL DEFAULT 'read_only' CHECK (mode IN ('read_only','read_write')),
    status        text NOT NULL DEFAULT 'pending' CHECK (status IN ('pending','active','error','
    disabled')),
    created_at    timestamptz NOT NULL DEFAULT now(),
    updated_at    timestamptz NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, name)
);

CREATE TABLE collections (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    connection_id  uuid NOT NULL REFERENCES connections(id) ON DELETE CASCADE,
    native_name    text NOT NULL,
    display_name   text NOT NULL,
    exposed        boolean NOT NULL DEFAULT false,   -- nothing exposed until reviewed
    UNIQUE (connection_id, native_name)
);

CREATE TABLE fields (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    collection_id  uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
    native_name    text NOT NULL,
    canonical_type text NOT NULL CHECK (canonical_type IN
                     ('string','text','integer','decimal','boolean','datetime','date',
                      'json','array','reference','binary','geopoint')),
    required       boolean NOT NULL DEFAULT false,
    is_primary_key boolean NOT NULL DEFAULT false,
    validation     jsonb NOT NULL DEFAULT '{}'::jsonb,
    ui             jsonb NOT NULL DEFAULT '{}'::jsonb,
    lossy          boolean NOT NULL DEFAULT false,   -- native type did not map cleanly
    UNIQUE (collection_id, native_name)
);

CREATE TABLE relations (
    id                   uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    source_collection_id  uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
    target_collection_id  uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
    name                 text NOT NULL,
    kind                 text NOT NULL CHECK (kind IN ('one_to_one','one_to_many','many_to_one',
    'many_to_many')),
    mapping              jsonb NOT NULL,        -- which fields join, or reference path
    UNIQUE (source_collection_id, name)
);

CREATE TABLE roles (
    id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id  uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    name       text NOT NULL,
    UNIQUE (tenant_id, name)
);

CREATE TABLE user_roles (
    user_id  uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id  uuid NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

CREATE TABLE permissions (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    role_id        uuid NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
```

```
    collection_id  uuid NOT NULL REFERENCES collections(id) ON DELETE CASCADE,
    action         text NOT NULL CHECK (action IN ('read','create','update','delete')),
    field_rules    jsonb NOT NULL DEFAULT '{}'::jsonb,  -- hide / mask / read-only per field
    row_filter     jsonb,                               -- portable filter AST applied to every
    query
    UNIQUE (role_id, collection_id, action)
);

-- Append-only audit log.
CREATE TABLE audit_log (
    id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tenant_id      uuid NOT NULL REFERENCES tenants(id),
    actor_id       uuid REFERENCES users(id),
    connection_id  uuid,
    collection     text,
    record_id      text,
    operation      text NOT NULL,       -- read, create, update, delete, connection.create,
    secret.read, permission.change, ...
    diff           jsonb,               -- redacted before/after
    ip             inet,
    request_id     text,
    at             timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX audit_log_tenant_time ON audit_log (tenant_id, at DESC);
CREATE INDEX connections_tenant ON connections (tenant_id);
CREATE INDEX collections_connection ON collections (connection_id);

-- Make the audit log append-only for the application role.
-- (Run as owner; replace app_role with the real application role.)
-- REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM app_role;

-- Optional: row-level security so a request can only see its own tenant.
-- ALTER TABLE connections ENABLE ROW LEVEL SECURITY;
-- CREATE POLICY tenant_isolation ON connections
--   USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

6.7 Query AST JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://sethucms.dev/schemas/query-ast.schema.json",
  "title": "SethuCMS portable query AST",
  "type": "object",
  "required": ["collection"],
  "additionalProperties": false,
  "properties": {
    "collection": { "type": "string", "minLength": 1 },
    "filter": { "$ref": "#/$defs/filter" },
    "sort": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["field"],
        "additionalProperties": false,
        "properties": {
          "field": { "type": "string" },
          "dir": { "enum": ["asc", "desc"], "default": "asc" }
        }
      }
    },
    "projection": { "type": "array", "items": { "type": "string" } },
```

```
    "page": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "limit": { "type": "integer", "minimum": 1, "maximum": 500, "default": 50 },
        "cursor": { "type": ["string", "null"] }
      }
    }
  },
  "$defs": {
    "filter": {
      "oneOf": [
        { "$ref": "#/$defs/comparison" },
        {
          "type": "object", "required": ["and"], "additionalProperties": false,
          "properties": { "and": { "type": "array", "minItems": 1, "items": { "$ref": "#/$defs/
    filter" } } }
        },
        {
          "type": "object", "required": ["or"], "additionalProperties": false,
          "properties": { "or": { "type": "array", "minItems": 1, "items": { "$ref": "#/$defs/
    filter" } } }
        },
        {
          "type": "object", "required": ["not"], "additionalProperties": false,
          "properties": { "not": { "$ref": "#/$defs/filter" } }
        }
      ]
    },
    "comparison": {
      "type": "object",
      "required": ["field", "op"],
      "additionalProperties": false,
      "properties": {
        "field": { "type": "string" },
        "op": {
          "enum": ["eq", "ne", "lt", "lte", "gt", "gte", "in", "nin",
                   "contains", "startsWith", "exists"]
        },
        "value": {}
      }
    }
  }
}
```

7 Security

Security drives the architecture. This section lists the controls the MVP must have.

7.1 Threat model

Threat	Control
Stolen credentials from the control plane	Envelope encryption; key held in KMS, not in the database
Credentials leaking to clients	Clients only call the API; secrets never leave the vault
SSRF through bring-your-own-database hosts	Block private ranges and metadata endpoints unless explicitly configured
Query injection	Parameterised queries, identifier allowlist from introspected metadata, operator allowlist
Over-broad database access	Dedicated least-privilege role, read-only by default
Cross-tenant impact	Isolated workers, per-connection pool and rate limits
Unaccountable changes	Append-only audit log outside tenant databases

7.2 Credentials

- Encrypt connection secrets with a data key wrapped by a KMS key (envelope encryption), with a documented rotation path.
- Back up the key separately from database backups. Losing it makes secrets unrecoverable.
- Prefer identity-based access over stored passwords: AWS IAM database auth, cross-account roles with external ID and Secrets Manager ARNs; Google service accounts, the Cloud SQL connector and IAM database auth; Azure Entra ID tokens and managed identities (tokens expire, so the vault refreshes them).
- Never log secrets. Redact connection strings in errors.

7.3 Least privilege

- Ask tenants for a dedicated CMS role scoped to the needed schemas or tables.
- Connections are **read-only by default**; write mode is an explicit opt-in.
- **Firebase:** the Admin SDK uses a service account with full access and is not gated by Security Rules. Scope the service account with IAM and enforce authorization in the CMS.
- **Supabase:** the service-role key bypasses row-level security. To enforce RLS, forward the editor's JWT or use a dedicated Postgres role.

7.4 Network controls

- Require TLS (`verify-full` where supported); allow custom CA bundles.
- Publish static egress IPs so tenants can allowlist the platform.
- Support SSH tunnels and private networking.
- Resolve and validate hostnames server-side and re-check after DNS resolution to avoid rebinding.

7.5 Injection defence

- Compile the query AST to parameterised queries only.
- Allowlist table and column names against introspected metadata.
- MongoDB: reject operator injection (`$where`, `$function`, keys starting with `$` from user input).
- Firestore and DynamoDB: validate field paths.
- Raw-query features sit behind a separate permission and are always audited.

7.6 Access control, audit and testing

SSO/OIDC for editors and MFA for admins. RBAC at collection level plus field rules (hide, mask, read-only) and row filters, enforced in the gateway independently of database grants. The audit log is append-only and stored in the control plane; it records actor, tenant, connection, collection, record id, operation, a redacted diff, IP and timestamp, and also logs connection changes, credential reads by workers and permission changes.

The security test suite covers SQL and NoSQL injection, identifier allowlist bypass, SSRF (private IPs, metadata IPs, DNS rebinding), permission bypass, secret leakage in logs and errors, and cross-tenant access. CI generates an SBOM, pins dependencies and scans them on every pull request.

8 API

One contract for every client. REST is the primary surface; GraphQL is generated from the same content model; OpenAPI is the source for SDK generation.

8.1 REST endpoints (v1)

Method	Path	Purpose
POST	`/v1/connections`	Create a connection (read-only by default)
GET	`/v1/connections`	List connections
GET	`/v1/connections/{id}`	Connection detail and health
PATCH	`/v1/connections/{id}`	Update config, mode, secret reference
DELETE	`/v1/connections/{id}`	Remove connection and secret
POST	`/v1/connections/{id}/introspect`	Start an introspection job
GET	`/v1/connections/{id}/collections`	List collections and fields
PATCH	`/v1/collections/{id}`	Edit display name, UI hints, exposure

Method	Path	Purpose
GET	`/v1/connections/{id}/collections/{name}/records`	List records
GET	`.../records/{rid}`	Get one record
POST	`.../records`	Create
PATCH	`.../records/{rid}`	Update
DELETE	`.../records/{rid}`	Delete
POST	`/v1/connections/{id}/batch`	Batch writes
GET	`/v1/audit`	Query the audit log
GET	`/v1/connections/{id}/counts`	Record counts for every collection
GET	`.../collections/{name}/count`	Record count for a filter
PUT	`.../drafts/{draftId}/schedule`	Schedule a publish
DELETE	`.../drafts/{draftId}/schedule`	Cancel the schedule
GET	`/v1/admin/origins`	List browser sites allowed (CORS)
POST	`/v1/admin/origins`	Allow a browser site
DELETE	`/v1/admin/origins/{id}`	Stop allowing a browser site
GET	`/v1/health`	Liveness and readiness

Write endpoints return `READ_ONLY` on read-only connections.

8.2 Counting records

`GET .../collections/{name}/count?filter=...&max=1000` answers `{"data": {"count": 1000, "exact": false}}`. The gateway counts by paging through the keys with the caller's own permissions, so a count never includes a record that a list would hide, and no adapter has to change. Counting stops at `max` (default 1000, at most 10000); `exact: false` means "at least". The answer is kept for 30 seconds and forgotten when a record is created or deleted through the gateway. `GET /v1/connections/{id}/counts` returns the same for every collection the caller can read; a collection whose count fails shows `null` and does not stop the others.

8.3 Singleton collections

A collection that holds exactly one record (site settings, a home page) is marked `singleton: true` in the schema file. The admin app opens that record directly, with no list, no New and no Delete. The gateway enforces it for every client: a second record is refused with `409` (`details.reason: "singleton"`), and so is deleting the record or starting a draft for a new one. The check is not atomic, so two creates at the very same moment on an empty collection could both pass; add a unique rule in the database if that matters.

8.4 Pagination, errors and versioning

Pagination is cursor-only. Cursors are opaque, signed and bound to the query, so a client cannot alter the filter between pages. Offset pagination is offered only where the adapter supports it.

Responses carry `capabilityNotes` listing any emulated or degraded behaviour.

Error code	HTTP	Meaning
`VALIDATION_FAILED`	400	Invalid input
`AUTH_FAILED`	401	Missing or invalid token
`FORBIDDEN`, `READ_ONLY`	403	Not permitted
`NOT_FOUND`	404	Unknown resource
`CONFLICT`	409	Write conflict
`CAPABILITY_UNSUPPORTED`	422	Adapter cannot do this
`RATE_LIMITED`	429	Too many requests
`CONNECTION_FAILED`	502	Database unreachable
`TIMEOUT`	504	Database timed out
`INTERNAL`	500	Unexpected error

The API is path-versioned (`/v1`); additive changes do not bump the version. Authentication uses a bearer token from the OIDC provider; service tokens are tenant- and role-scoped, rotatable and never accepted from browsers. Writes accept an `Idempotency-Key` header, and rate limits apply per tenant and per connection.

9 SDKs

Every client stack uses the same API. SDKs are generated from `openapi.yaml` so they never drift, with a small hand-written layer for ergonomics.

Language	Package	Approach
TypeScript / JavaScript	`@sethucms/sdk`	Hand-tuned, first class
Dart / Flutter	`sethucms` (pub.dev)	Generated models plus hand-written client
Python	`sethucms` (PyPI)	Generated models plus hand-written client
Kotlin / Java, Swift	generated	Android, JVM, iOS
Go, C#, PHP, Ruby, Rust	generated	On demand

Check package names on npm, pub.dev and PyPI before committing to them.

9.1 Hand-written layer

Each first-party SDK adds the same small set of behaviours: a *client* (base URL, token, request pipeline), *auth* (token refresh), a *query builder* in the language's idiom, *pagination* as an async iterator over cursors, optional *realtime* helpers, and typed *errors* from the API error codes.

9.2 TypeScript SDK 0.2.0

Version 0.2.0 adds `count()` on a query, `counts(connectionId)` on the client, and `drafts` on a collection with `create`, `schedule`, `cancelSchedule` and `publish`. A scheduled draft publishes its latest saved values with the scheduler's permissions; on a serverless gateway `schedule` fails with `CAPABILITY_UNSUPPORTED`. The release only adds methods, so code written for 0.1.0 keeps working. It is on npm.

9.3 Rules

1. SDKs never hold database credentials. They only hold API tokens.
2. Browser and mobile SDKs use short-lived user tokens, never service tokens.
3. CI regenerates SDKs on every API release and fails if generated code is out of date.
4. Semantic versioning follows the API version.

10 Deployment

Built today: one container that runs the gateway and serves the admin app on a single port, with its state in one volume (see the next sections). This section describes the larger target design: separate api and worker pods, a queue and a managed control-plane database. The worker, the queue and the shared control-plane database are not built yet, and the Kubernetes and Terraform files in `sethucms-infra` follow the larger design.

10.1 Environments and services

Service	Scaling	Notes
api	Horizontal, stateless	Never connects to customer databases directly
worker	Horizontal, per-connection pool caps	Only component that decrypts secrets
admin	Static assets	Talks only to the API
control-plane DB	Managed Postgres	Backups, point-in-time recovery
queue	Managed	Introspection jobs, realtime fan-out

Environments

local (`docker compose` with api, worker, admin, control-plane Postgres and test databases), *staging* (full stack, synthetic tenant data) and *production* (Kubernetes, managed Postgres, KMS-backed secrets).

10.2 Cloud identity and networking

Cloud	Identity (avoid stored passwords)	Private networking
AWS	IAM database auth, cross-account role with external ID, Secrets Manager ARN	PrivateLink, VPC peering
Google Cloud	Service accounts, Cloud SQL connector, IAM database auth	Private Service Connect
Azure	Entra ID tokens, managed identities (refreshed by the vault)	Private Link
Any	Passwords in the vault, SSH tunnels	Static egress IPs

10.3 Configuration, observability, CI/CD

Configuration uses environment variables (names only, never committed values): `DATABASE_URL`, `KMS_KEY_REF`, `OIDC_ISSUER`, `OIDC_CLIENT_ID`, `SESSION_SECRET_REF`, `EGRESS_BLOCKLIST_EXTR`

`A`, `WORKER_MAX_POOL_PER_CONNECTION`, `LOG_LEVEL`. In production, secrets come from a secrets manager, not from `.env` files.

Observability: structured logs with request ids and secrets redacted at the logger; per-connection metrics for latency, errors, pool usage and rate-limit hits; alerts on failed introspection, credential refresh failures and unusual audit patterns.

The CI/CD pipeline runs lint, type-check and unit tests; the conformance suite per adapter against containerised databases; the security suite; image builds with SBOM and dependency scanning; SDK regeneration with drift checks; then deploys to staging, runs end-to-end tests and promotes.

Control-plane backups use automated backups and point-in-time recovery. The KMS key is backed up separately; without it, stored secrets cannot be decrypted even from an intact backup. Run a key-rotation drill before launch.

11 Gateway

The gateway (`sethucms/apps/api`) is the one process between every client and every database. It holds credentials, applies permissions, keeps drafts and writes the audit log. Clients never talk to a database.

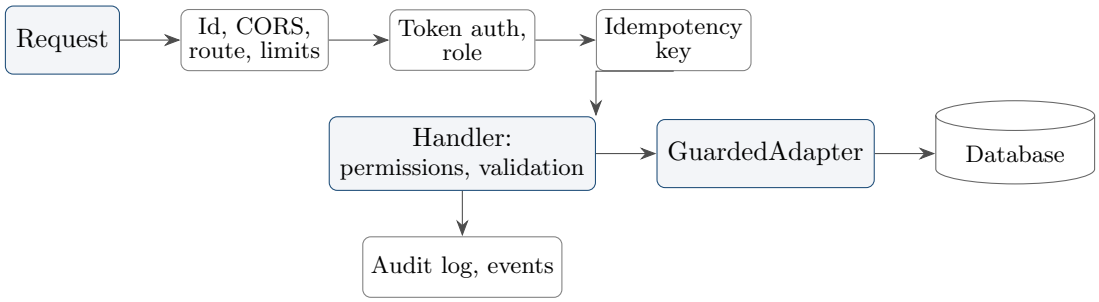


Figure 4: Request pipeline. A failure at any step becomes the standard error body; database text never leaves the server.

11.1 Many databases at once

A connection is a saved row: name, adapter type, mode, settings, encrypted credentials and the collections it exposes. The registry opens an adapter on first use and keeps it open, so memory, PostgreSQL and further connections run side by side, each with its own pool, capabilities and content model. A new connection is read-only and every collection is hidden until an administrator exposes it.

11.2 Security controls

Control	How it works
API tokens	Roles admin, editor, viewer. Only SHA-256 hashes are stored; comparison is timing-safe
Vault	AES-256-GCM, bound to the connection or draft id so ciphertext cannot be moved. Protects database credentials and drafts
Secrets	Vault key, cursor secret and preview secret are required in production; generated into an owner-only file locally
Network guard	Cloud metadata and link-local addresses always blocked; private ranges blocked unless allowed
Rate limits	Per address and per token
Idempotency	Repeating a write with the same `Idempotency-Key` returns the first answer
Audit	Append-only JSON lines with secrets redacted
Admin page	Served with a strict content security policy

11.3 Browser access (CORS)

A website can call the gateway from a browser only if its address is allowed. Requests carry the token in a header and never use cookies, so the gateway never sends `Access-Control-Allow-Credentials`, and allowing a site gives it no access by itself. Addresses come from `SETHUCMS_ALLOWED_ORIGINS` (exact, fixed) and from the admin screen *CORS origins* (admin only, every change audited). With `SETHUCMS_ALLOW_WILDCARD_ORIGINS=1` an admin can add one-label https wildcards such as `https://*.preview.example.com`; shared hosting domains such as `vercel.app` are always refused. The screen also lists sites that were just refused, with an *Allow this site* button (kept in memory, at most 20, empty after a restart). Preflight `OPTIONS` is answered before login, and every answer carries `Vary: Origin`. A console message "blocked by CORS policy" or a `Failed to fetch` with no status usually means the site is not on the list; `401` and `403` are not CORS problems.

11.4 Counts and scheduled publishing

The admin shows a record count beside each collection; the gateway counts with the caller's permissions, capped at 1000 by default. A timer inside the gateway looks every 15 seconds for scheduled drafts that are due and publishes them. It runs only in a gateway that stays running, so it is off in serverless mode.

11.5 Known limits

Single process per data folder (file state, per-process rate limits and events); three fixed roles; no single sign-on; events cover only changes made through the gateway; a short DNS-rebinding window in the network guard; adapters run in-process, so gRPC adapters are not wired yet.

12 Drafts, Publishing and Live Preview

Editing never touches the database. Every edit is a draft stored encrypted in the gateway. The database changes only when someone presses Publish, through the same checked write path as

any other change.

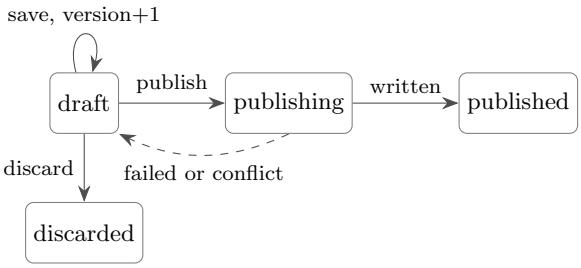


Figure 5: Draft life cycle.

12.1 What the editor sees

Step	Admin app	Gateway
Start typing	"Saving draft", then "Draft saved"	Creates the draft, version 1
Keep typing	Saves 0.7 s after the last key	Replaces the draft, version + 1
Preview	Pane updates as you type	Streams the draft merged over the published record
Publish	"Published"	Validates, checks for conflicts, writes, removes the draft
Schedule	Pick a date and time under the form	Stores the time in UTC on the draft; the editor stays open
Discard	Draft disappears	Marks discarded; database untouched
Record changed elsewhere	Conflict panel	Refuses with `409 record_changed`

12.2 Three protections

1. **Version check.** A save states the version it started from; a stale save gets `409` and nothing is lost.
2. **Fingerprint check.** A draft of an existing record stores a hash of that record. If the row changed since (another editor, or a direct database change), publish stops and shows the difference. The editor reloads, or chooses Publish anyway (`force`, recorded in the audit log).
3. **Claim before write.** Publish first moves the draft to `publishing`, so a double click or retry cannot write twice.

12.3 Scheduled publishing

An editor can choose a time instead of pressing Publish. The time must include its zone (`Z` or `+05:30`), be in the future and at most 366 days ahead. At that time the latest saved values are published with the permissions of the person who scheduled it; editing the draft afterwards keeps the schedule. The gateway looks for due drafts every 15 seconds and once at start, so a draft that came due while the gateway was off publishes when it starts again. If publishing fails (the live

record changed, the values no longer validate, the person lost access), nothing is published, the schedule is cleared, the reason is kept as `scheduleError` and it never retries by itself. The audit log records `draft.schedule`, `draft.unschedule` and `draft.schedule.fail`; a success is the normal `draft.publish`. With `SETHUCMS_SERVERLESS=1` the call is refused with `422 CAPABILITY_UNSUPPORTED`. With several gateways sharing PostgreSQL state the publish is claimed first, so only one publishes; one live test with two gateway instances in the same process on PostgreSQL 16 passed, and it has not been run across separate machines.

12.4 Storage

Drafts live in the gateway's state store, never in the customer database, encrypted with AES-256-GCM bound to the draft id. Read-only connections can hold drafts, but publish is refused on them. Deleting the gateway data folder deletes unpublished drafts; published content stays in the customer database.

12.5 Live preview

A preview link is `base64url(claims).hmac`. Claims name the draft and the token that created it, and expire after 15 minutes. `GET /v1/preview/:token` returns the published record with the draft applied, masked for the creator; `/events` streams the same payload whenever the draft changes. Revoking the creating token kills the link. A preview link cannot publish or read anything else.

12.6 API summary

Request	Purpose
`POST .../collections/:n/drafts`	Create a draft
`GET .../collections/:n/drafts`	List drafts
`PATCH .../drafts/:id`	Save (send `version`)
`DELETE .../drafts/:id`	Discard
`POST .../drafts/:id/publish`	Publish (`version`, optional `force`)
`PUT .../drafts/:id/schedule`	Schedule the publish (`publishAt` with a time zone)
`DELETE .../drafts/:id/schedule`	Cancel the schedule
`POST .../drafts/:id/preview-link`	Create a preview link
`GET /v1/preview/:token[/events]`	Read, then stream the preview

Only fields in the content model can be saved (unknown fields return `400`). Required-field errors appear at publish time so half-written entries can always be saved.

13 Build and Run

13.1 Requirements

Node.js 20 or newer and pnpm, with these repositories cloned side by side: `sethucms`, `sethucms-adapter-sdk`, `sethucms-adapters` and `sethucms-admin`. Until packages are published, they link to each other by folder, so the folder names matter.

13.2 Build and start

```
cd ~/sethucms
sh scripts/build-all.sh     # core, adapter-sdk, adapters, gateway, admin
sh scripts/run-local.sh     # http://127.0.0.1:8080
```

The first start prints an administrator token once; paste it into the sign-in page, or set `SETHUCMS_ADMIN_TOKEN` (32 to 200 characters) beforehand. Data (connections, drafts, tokens, audit log, generated secrets) is kept in `.sethucms-data/`; back it up and do not commit it.

13.3 Connect a database

Open Connections and add a connection; it starts read-only with all tables hidden. Expose the tables you want, switch to read-write if editing is needed, then edit under Content.

13.4 Tests

Suite	Result recorded
Core	29 tests passing
Gateway	78 tests passing (89 with PostgreSQL), including logging and migration
Live PostgreSQL	Gateway suite passing (injection safety, drafts, direct-change conflict, concurrent databases, read-only); migration suite passing against PostgreSQL 16
Admin app	Type check clean; 15 of 15 browser tests passing

Not yet verified: the build scripts under pnpm on a fresh machine, Docker images, the Dart SDK and the Go adapter server.

13.5 Production settings

Setting	Why
`NODE_ENV=production`	Refuses to start without secrets; turns off demo data and private networks
`SETHUCMS_VAULT_KEY`, `SETHUCMS_CURSOR_SECRET`, `SETHUCMS_PREVIEW_SECRET`	One random value each; keep in a secrets manager
`SETHUCMS_ADMIN_TOKEN`	First administrator; create others, then rotate it
`SETHUCMS_ALLOWED_ORIGINS`	Only the sites allowed to call the API
`SETHUCMS_TRUST_PROXY=true`	Only behind a proxy you control
TLS	Terminate HTTPS in front of the gateway

14 CI/CD Pipelines

Every repository has its own GitHub Actions workflows. They are kept in `sethucms/ci-workflows/` and copied into each repository's `.github/` folder with `sh scripts/install-ci.sh`,

because tools that edit files for you cannot write into `.github`. Each repository has a "CI OK" job that gathers the results into one check, which is the one to require in branch protection.

14.1 What runs where

Repository	On every push and pull request	On a `v*` tag
sethucms	Core and gateway tests on Node 20 and 22, gateway against live PostgreSQL, dependency audit, CodeQL	Builds one container image (gateway with admin inside), pushes it to the GitHub registry, scans it, creates a release
adapter-sdk	Type check, build, tests on Node 20 and 22	Packed package attached to a release
adapters	Same, plus live PostgreSQL	Packed packages attached to a release
admin	Type check, tests (Node 22), the three builds (separate address, embedded, content-only), check that fonts are bundled, CodeQL	Built app attached to a release
sdk-ts	Type check, build, tests on Node 20 and 22	Packed package attached to a release; the tag must match the version in `package.json`
sdk-python	Tests on Python 3.9 to 3.13, package build	Wheel and source package attached
sdk-dart	Format, analyze, test	Release created
adapter-template-python	Generated protocol code is up to date, tests on Python 3.10 to 3.13	—
adapter-template-go	Generate protocol code, vet, test with the race detector, build	—
infra	Compose, Kubernetes and Terraform checks, configuration scan, Dockerfile lint	—
docs	Link check, schema check, PDF build	PDF attached to a release

Dependabot is switched on in every repository for its package ecosystem and for the workflow actions themselves. The admin repository also has a deploy workflow: on a push to `main` it type checks, runs the tests, deploys to Vercel and checks the live site. It needs the repository secrets `VERCEL_TOKEN`, `VERCEL_ORG_ID` and `VERCEL_PROJECT_ID`; without them a push only prints a warning and skips the deploy, while a manual run fails.

14.2 How the linked repositories are built

Until the packages are published, the repositories link to each other by folder. A workflow therefore checks out its own repository and the ones it needs side by side, then builds them in

dependency order: core, adapter SDK, adapters, gateway, admin. If the repositories are private, add a repository secret named `CROSS_REPO_TOKEN` with read access.

14.3 Release image

`sethucms/docker/Dockerfile.gateway` builds from the folder that holds the four repositories. The image runs as a non-root user, keeps data in a `/data` volume, has a health check, and refuses to start in production mode without the three secrets. The build records provenance and a software bill of materials. The image still contains development dependencies, so it is larger than it needs to be.

14.4 Not covered yet

Deployment to a cloud (Terraform apply, Kubernetes rollout) needs your cloud credentials and is not written. The npm packages are published by hand with `npm publish`; no workflow publishes them, and nothing is published to PyPI or pub.dev yet. The workflows install with `--no-frozen-lockfile` until lockfiles are refreshed and committed. They run on GitHub on every push and pull request.

14.5 Tooling decision: Node and pnpm

The standard is Node 22 with pnpm 9. Bun starts the gateway and passes the unit tests I tried, so it is fine for local use, but it is not the supported path: the folder links, the lockfiles, the workflows and the Docker build were all built and tested on pnpm. Moving to Bun would mean re-testing all of them.

15 Deploy to Production

This is the checklist for running the gateway and admin app for real use. The steps follow the code and the Dockerfile as written. The gateway and its production checks are tested; the container image, the compose and Kubernetes files and the workflows have not been run end to end, so do the verify steps before trusting them.

15.1 What you deploy

One container runs the gateway and serves the built admin app from the same address: one port (8080), one URL, no extra CORS setup. Put a reverse proxy or load balancer in front for HTTPS. A volume mounted at `/data` holds connections, drafts, tokens and the audit log. The gateway reaches your databases from the container.

15.2 Steps

1. **Build the image.** From the folder that holds the four repositories: `docker build -f sethucms/docker/Dockerfile.gateway -t sethucms .`, or push a `v*` tag and let GitHub build it.
2. **Create the secrets** once and keep them in a secrets manager: vault key, cursor secret and preview secret (`openssl rand -base64 32` each) and the first admin token (`openssl rand`

`-hex 32`). Back up the vault key separately. Without it, saved database passwords and drafts cannot be decrypted.

3. **Run it** with the volume on `/data`, the four secrets as environment variables, and `SETHUCMS_TRUST_PROXY=true` only when a proxy you control is in front.
4. **Verify.** `/v1/health` and `/v1/ready` answer; sign in; add a test connection, expose a collection, create a draft, preview it, publish; confirm Audit shows the publish.

In production mode the gateway refuses to start if a secret is missing, turns off the demo connection, and turns off private-network database access unless asked.

15.3 Day-two operations

Task	How
Back up	The `/data` volume and the secrets; test a restore
Rotate the admin token	Create a new admin token under Access, sign in with it, revoke the old one
Upgrade	Start the new image on the same volume and secrets, run the verify steps, keep the old image for rollback
Logs	One JSON line per request on standard error (see Logging and Monitoring), secrets removed; alert on 5xx and repeated 401 or 429
Database users	Give each connection the fewest rights you can; keep it read-only until editing is needed

15.4 Limits to plan around

One gateway process per data volume (state is a file; rate limits, idempotency keys and live events are per process), so do not run two replicas on the same data. Three fixed roles and no single sign-on. Live events show only changes made through the gateway. The image still contains development dependencies. Deploy workflows (Terraform apply, Kubernetes rollout) are not written.

15.5 Checklist for an agent doing a deploy

Read `AGENTS.md`; confirm the tests pass; build the image and confirm it refuses to start with a secret missing; confirm health and ready; never print or commit secrets; do not run `git commit`, `git push`, `terraform apply` or change a live environment unless the owner said to; report exactly what was verified and what was not.

16 Logging and Monitoring

Everything here is built into the gateway and tested; no extra package is needed. Operational logs go to **standard error** (the start-up banner is the only thing on standard output), as one JSON object per line, or readable text at a terminal. The audit log is a separate file, `audit.log`, in the data folder.

16.1 What is logged

Each request writes one line with a request id (also returned in the `X-Request-Id` header and in every error), a trace id when a W3C `traceparent` header is sent, the method, the path without its query string, the route pattern, the status, the time taken, the caller and role. The caller's IP is added only to 401, 403 and 429 answers. Slow requests (default over one second) and 401, 403 and 429 answers are warnings; 5xx answers are errors; health checks are debug lines. Unexpected failures write an error line with the reason, while the caller only sees a generic `INTERNAL` error. Request bodies, query strings, record values, tokens and passwords are never logged on purpose, and every field also passes a redaction filter.

Setting	Meaning
`SETHUCMS_LOG_LEVEL`	debug, info, warn or error (default info)
`SETHUCMS_LOG_FORMAT`	json or pretty (default pretty at a terminal outside production, else json)
`SETHUCMS_SLOW_REQUEST_MS`	Requests slower than this are warnings (default 1000)
`SETHUCMS_AUDIT_MAX_MB`, `SETHUCMS_AUDIT_KEEP`	Rotate the audit log at this size; rotated files kept (defaults 10 and 5)
`SETHUCMS_METRICS_TOKEN`	Lets a scraper read metrics without an admin token

16.2 Metrics

`GET /v1/metrics` answers in the Prometheus text format and needs an admin token or the metrics token. It reports requests by method, route and status, request duration, in-flight requests, errors by code, rate-limited requests, change events, open event streams, connections by adapter, active tokens, audit entries written, migration records by result, event-loop delay, memory, CPU and uptime. Labels are bounded: routes are patterns, never raw paths, and each metric stops adding label combinations after a fixed limit. Worth alerting on: a 5xx rate above two percent, a 95th percentile above one second, repeated 401 or 429, any failed migration records, and `up` equal to zero.

16.3 The audit log

Every change to connections, content, tokens and drafts is appended to `audit.log`. Each entry carries the hash of the previous entry, so editing, removing or reordering a line breaks the chain. `GET /v1/admin/audit/verify` checks it and reports where it first breaks. Tampering is shown, not prevented: someone with write access to the volume can rebuild the chain, so keep a copy somewhere the gateway cannot write. The log rotates by size and the chain continues across files.

16.4 Limits

Logs and metrics are per process. The ready check confirms the process is up and does not test each database. Tracing is correlation only: the trace id is copied into logs and no spans are exported. The alert rules are examples and were not tested against a live Prometheus.

17 Moving Data Between Databases

An administrator can copy the records of one connection into another: PostgreSQL to a new PostgreSQL, a demo into a real database, or between any two adapters the gateway supports. It is on the *Move data* page of the admin app and under `/v1/admin/migrations` in the API.

17.1 How it works

1. **Plan** (dry run). The gateway reads both databases' structure and decides how each collection maps. Nothing is written. The plan lists blocking problems, warnings, record counts and how ids are handled.
2. **Run.** The copy runs in the background, one migration at a time. Collections that others point at are copied first. Each record is checked against the target's own rules, then written. Progress can be followed and the run cancelled.
3. **Check.** Record counts are compared before and after, and each stored record is compared with what was sent.

The source is only read, the target must be read-write, and only administrators can start a migration. Record values are never logged, audited or put in error messages. By default records that already exist are left alone, so a stopped or failed run is continued by running it again; `overwrite` replaces them and `fail` reports them.

17.2 What the plan blocks

The run is refused when the target has no matching collection or field, when a source field has nowhere to go (unless `dropUnmappedFields` is set), when a required target field is not filled, when types cannot hold each other's values, or when a collection points at itself while the target renumbers records. Compatible but different types (string and text, integer into decimal) are warnings, and each value is checked.

17.3 Ids and references

If the target key is an ordinary key, ids are copied as they are. If the target numbers its own records (a PostgreSQL serial column), the target assigns new ids by default and every reference in later collections is rewritten to match; the target collection must then be empty, because running again would duplicate everything. With `preserveIds` the original ids are written into the generated key. PostgreSQL accepts this, after which its sequence must be moved past the largest id; a database that ignores the id makes the run fail with a clear message. References are found from a `references` marker on a field or from a many-to-one relation to the other collection's key, which is how PostgreSQL foreign keys are reported.

17.4 What it does not do

It copies records only. It does not create tables or collections in the target, copy indexes, constraints, views or users, run in one transaction (a failure leaves a partial copy that repeating completes), or freeze the source while it runs. Job progress is kept in memory and is lost if the gateway restarts. For a live system: back up, run once, stop writes, run again.

17.5 Verified

Memory to memory; PostgreSQL to PostgreSQL with ids kept, and renumbered with foreign keys rewritten; memory into PostgreSQL; and PostgreSQL into memory with ids kept, which fails clearly. The PostgreSQL runs used PostgreSQL 16. Not tested: other PostgreSQL versions, very large tables, a live workload. Only the memory and PostgreSQL adapters exist, so migration works between those two for now.

18 Database adapters

The gateway ships with these adapters. Each one is a separate package in `sethucms-adapters`, passes the same conformance suite, and is checked by the same guard layer (field allowlist, parameters only, permission engine).

Adapter	Kind	Package	Driver	Tested against
Memory	`memory`	`@sethucms/adapter-memory`	none	in-process
PostgreSQL	`postgres`	`@sethucms/adapter-postgres`	`pg`	PostgreSQL 16
SQLite	`sqlite`	`@sethucms/adapter-sqlite`	Node's built-in `node:sqlite`	real SQLite files
MySQL / MariaDB	`mysql`	`@sethucms/adapter-mysql`	`mysql2` (peer)	**MariaDB 10.11** and **MySQL 8.0** (real servers, full conformance suite) and Dolt
MongoDB	`mongodb`	`@sethucms/adapter-mongodb`	`mongodb` (peer)	**FerretDB 1.24** (MongoDB-compatible), not MongoDB itself
Firestore	`firestore`	`@sethucms/adapter-firestore`	`@google-cloud/firestore` (peer)	an **in-memory fake**, not Firestore

Read the "Tested against" column before you rely on an adapter. A compatible server is not the real thing: MySQL, MariaDB, MongoDB and Firestore have not been run through these tests, and you should run the live tests (below) against your own server before production use.

Peer drivers are not installed for you. Add the one you need to the gateway (`pnpm add mysql2` in `apps/api`).

18.1 Shared rules

- Passwords and keys go in the connection's credentials, which the vault encrypts. They never appear in settings, logs or error messages.
- Network connections go through the gateway's network guard (private ranges blocked in production unless `SETHUCMS_ALLOW_PRIVATE_NETWORKS=true`).
- Connections can be read-only. Writes are then refused before reaching the driver.
- Every adapter can *create* a missing collection when a migration asks for it (doc 17).

18.2 SQLite

For small sites, edge deployments and local work. A "connection" is a **file on the gateway's disk**.

Setting	Meaning
`path`	`:memory:` or a file name **inside** the folder set by `SETHUCMS_SQLITE_DIR`
`busyTimeoutMs`	how long to wait for a lock (0-60000, default 5000)

- File databases are off until `SETHUCMS_SQLITE_DIR` is set. Paths with `..`, absolute paths outside the folder, and URIs are refused.
- Needs **Node 22.5 or newer** (`node:sqlite`). The Docker image must use Node 22.
- SQLite allows one writer at a time. Do not point another program at a file the gateway is writing unless you accept lock waits.
- Back up with the SQLite backup API or by copying the file while the gateway is stopped. Copying a live file can give a torn copy.

18.3 MySQL and MariaDB

Setting	Meaning
`host`, `port`	server address (port default 3306)
`database`	database name
`ssl`	default **true**; turn off only on a trusted private link
`maxConnections`	1-20, default 5
`statementTimeoutMs`	100-120000, default 15000
credentials `user`, `password`	encrypted in the vault

- Identifiers are quoted with backticks and values are always parameters.
- Pagination is offset based.
- `LIKE` wildcards in user text are escaped.
- Not verified on MySQL 5.7 or 9, or other MariaDB versions. MariaDB 10.11 and MySQL 8.0 are verified. It has no real JSON type (JSON is LONGTEXT plus a `json_valid` check), and the adapter finds those columns from the check; a bug where they came back as text was found and fixed that way.

18.4 MongoDB

MongoDB has no schema, so the adapter works out collection shapes by **sampling documents**, or you declare them.

Setting	Meaning
`host`, `port`, `srv`	address; `srv` `true` for Atlas `mongodb+srv` (no port)
`database`, `authSource`	database, and where the user is defined (default `admin`)
`tls`	default **true**
`sampleSize`	documents sampled per collection (1-1000, default 100)
`collections`	declared definitions that replace sampling
`maxConnections`, `statementTimeoutMs`	as above

- Integer ids are issued from a counter collection `_sethucms_counters`; declared and discovered shapes are kept in `_sethucms_schemas`. Both are hidden from content.
- Queries are built as typed documents from the allowlisted filter tree. Operators in user values (`$ne`, `$where`) are rejected before they reach the driver.

- Sampling can miss rare fields. Declare the collections you care about.
- Tested against FerretDB 1.24 only. Real MongoDB behaviour (collation, index use, transactions) is not verified.

18.5 Firestore

Setting	Meaning
`projectId`, `databaseId`	project and database (default `(default)`)
`emulatorHost`	`host:port` of the emulator, for development only
`sampleSize`, `collections`	as for MongoDB
`maxScanDocuments`	most documents a text search may read (10-50000, default 5000)
credentials `serviceAccount`	service account key JSON, encrypted in the vault; otherwise Google default credentials

- Firestore cannot do OR with inequality, several inequality fields, or arbitrary text search. The gateway **refuses** such queries (`CAPABILITY_UNSUPPORTED`) instead of approximating them.
- `contains` and `startsWith` on text are done by reading up to `maxScanDocuments` documents; they are slow and bounded on purpose.
- Integer ids come from `_sethucms_counters`, as for MongoDB.
- **The content adapter was only tested against an in-memory fake that we wrote.** (The gateway's shared state store, a different piece of code, is also tested against the Firestore emulator; see doc 26.) The adapter has never talked to Firestore or the emulator. Treat this adapter as experimental until you have run it against the emulator or a test project.

18.6 Running the live tests against your own server

From `sethucms-adapters`:

```
SETHUCMS_TEST_PG_URL=postgres://user:pass@host:5432/db  pnpm --filter @sethucms/adapter-postgres
    test
SETHUCMS_TEST_MYSQL_URL=mysql://user:pass@host:3306     pnpm --filter @sethucms/adapter-mysql
    test
SETHUCMS_TEST_MONGO_URL=mongodb://host:27017            pnpm --filter @sethucms/adapter-mongodb
    test
```

The tests create and drop their own scratch database or collections. Use a throwaway server. Without these variables the live tests are skipped.

18.7 Writing your own adapter

Implement the `Adapter` contract (doc 5) and run `defineConformanceSuite`. SQL databases should build on `@sethucms/adapter-sql-kit`, which holds the dialects, query builders, table creation and row normalising shared by PostgreSQL, MySQL and SQLite. Optional methods: `createCollection` (needed for migration table creation) and `finishBulkLoad` (for example, moving a sequence past copied ids).

19 Open-source release checklist

> Status: 0.1.0 is released. All ten packages are on npm under `@sethucms`, and the repositories are public. Keep this list for the next release and for the items still open: branch protection, community-health files, npm trusted publishing and two-factor on the owner account.

The code is MIT licensed (`LICENSE` in every repository). This list is what to do before the repositories go public. Items marked **owner** need a person; they are not things a coding agent should do.

19.1 Before making anything public

1. **Secrets scan, whole history.** Run a scanner over every repository's full git history (for example `gitleaks detect` or `trufflehog git file://.`). One leaked key in an old commit is public once the repository is. If anything is found, rotate the secret first, then clean the history. **owner**
2. **Read the code you are publishing.** Remove private notes, customer names, internal URLs and test data that is not synthetic.
3. **Licences of dependencies.** Run a licence report (`pnpm licenses list`) and check that nothing conflicts with MIT. Add the peer drivers (`mysql2`, `mongodb`, `@google-cloud/firestore`) to your list; they are not bundled.
4. **Name and trademark.** Search for "Sethu" and "SethuCMS" in the trade-mark registers of the countries you care about and in package registries and domain names. MIT gives no trademark rights; decide whether you want a trademark policy. **owner**
5. **Be honest in the README.** Keep the "tested against" table from doc 18. Say that MySQL, MongoDB and Firestore adapters have only been run against stand-ins, and that deploy workflows and the container image are unverified.
6. **Security contact.** Enable *private vulnerability reporting* in each repository (Settings, Code security). Check `SECURITY.md` names it. **owner**

19.2 The organisation profile

The `org-github/` folder holds the community files (code of conduct, contributing guide, issue and pull request templates, security policy). Publish it as a repository named `.github` inside the `sethucms` organisation. GitHub then applies those files to every repository that has none of its own. Some tools cannot write `.github` paths, so copy the folder yourself. **owner**

19.3 Day of release

1. Make repositories public in dependency order: `sethucms`, `sethucms-adapter-sdk`, `sethucms-adapters`, `sethucms-admin`, then the rest.
2. Turn on branch protection for `main`: pull request required, tests must pass, no force pushes.
3. Turn on Dependabot alerts and secret scanning (free for public repositories).
4. Publish packages to npm in the order given in `RELEASING.md` in the `sethucms` repository (core, adapter SDK, SQL kit, adapters, then the TypeScript SDK). The ten 0.1.0 packages were packed and installed together in a clean folder and all import; the shared conformance suite passes from the installed copies. After publishing, remove the `link:` overrides from `sethucms-adapter-sdk` and `sethucms-adapters` and check that a plain `npm install` works.
5. Tag a first release with notes that list what is verified and what is not.

19.4 After release

- Watch issues and security reports for the first weeks; reply within a few days even if only to say you saw it.
- Label a few small issues `good first issue`.
- Re-read doc 18 whenever an adapter is tested against a real server, and update its row.

20 Sponsorship and funding

SethuCMS is free, open-source software (MIT licence). Sponsorship is how people who use it can help keep it maintained. This page says how funding is set up, what it pays for, and what sponsors do and do not get.

This page is a plan and a set of rules, not legal or tax advice. Items marked **check** were not verified.

20.1 Where the money can go

Route	Fits now?	Notes
GitHub Sponsors on your personal profile (`kasinadhsarma`)	**Yes, start here**	GitHub lists India as a supported region. Signing up needs an Indian bank account. Sponsorships from personal accounts carry no GitHub fee. Sponsorships from organisation accounts can carry fees of up to 6% (card and service), lower if they pay by invoice.
GitHub Sponsors for the **SethuCMS organisation**	Not yet	GitHub requires an organisation to be legally operating in a supported region. A GitHub organisation alone, with no registered entity behind it, is not enough. Revisit if you later register one. **check** with GitHub's current rules when you do.
Open Collective with a fiscal host	Later	A host holds the money and does the paperwork for a project. Whether a host will take an India-based project without a registered entity, and the host's fee, are **check**.
Other tip jars (Liberapay, Ko-fi and similar)	Optional	Add only if people ask. Each one is another account to run and report.
Grants and programmes for open source	Later	Many require a project that is already used by real people, and some require a registered organisation. **check** each one.

20.2 Setting it up (you do these steps)

1. **GitHub Sponsors:** open `https://github.com/sponsors` and apply for your personal profile. GitHub will ask for payout details and tax information. These are your own financial details, so enter them yourself and never share them in chat or in an issue.

2. **Profile text:** say what SethuCMS is, what the money pays for (below), and that it is free software with no paid features.
3. **Tiers:** a one-time option and two or three monthly options are enough. Suggested shape (you decide the amounts):

Tier	What the sponsor gets
Backer	A thank-you, and a name in `SPONSORS.md` if they agree
Supporter	The same, and a name or logo in the README if they agree
Organisation sponsor	The same, plus a link to their site in the README

   Nothing in any tier changes what the software does or what anyone can use.
4. **Button:** put `org-github/FUNDING.yml` in the root of the organisation's `.github` repository so every repository shows a Sponsor button.
5. **Ask a chartered accountant** before you accept money: how sponsorship income is reported, whether foreign payments have any rules or limits, and when a registered entity would make sense. **check**

20.3 What the money pays for

- Test servers and services (real MySQL, MongoDB, Firestore and cloud databases for the adapter tests)
- A domain name and hosting for the docs
- An independent security review before a stable release
- Small fixed rewards for accepted contributions, if you choose to offer them

Publish a short summary of income and spending each quarter, even if it is two lines.

20.4 Rules for sponsors

- **Sponsors cannot buy decisions.** Money does not buy roadmap control, security exceptions, early access to vulnerabilities, or changes to the licence.
- **No access to anyone's data.** Sponsors never get access to a deployment, a database or logs.
- **No paid features.** The software stays free. Support requests from sponsors are answered in the same queue as everyone else's.
- **Names are opt-in.** A sponsor's name or logo appears in the README or `SPONSORS.md` only if they agree. A name can be personal data under GDPR, so remove it on request.
- **You may decline a sponsor** whose business or conduct conflicts with the project. Say so plainly and refund if the platform allows it.
- **Security reports are never for sale.** Reports go to the private channel in `SECURITY.md`, from sponsors or anyone else.

20.5 Before you ask for money

People sponsor software they trust. Before you announce this, the project should have a version someone else can install and run. Until the Docker image is verified and the adapters other than PostgreSQL and SQLite have been tried on real servers, ask for feedback first and sponsorship second.

20.6 Open questions for you

- Which amounts for the tiers?
- Do you want to offer contribution rewards, or keep all money for infrastructure and the security review?
- Do you plan to register an entity later (company, trust or society)? That decides whether to move to an organisation sponsor profile or a fiscal host.

21 Single sign-on (OpenID Connect)

People sign in to the admin app with your company login (Okta, Google Workspace, Microsoft Entra, Keycloak, Auth0 and so on). The gateway never sees their password.

Status: tested against a small fake identity provider written for the tests, not against a real Okta, Google or Entra. Try yours in a staging setup first.

21.1 How it works

1. The admin app shows "Sign in with single sign-on" when `GET /v1/auth/config` says it is on.
2. The browser goes to `/v1/auth/oidc/login`, then to your provider (authorization code flow with PKCE).
3. The provider sends the browser back to `/v1/auth/oidc/callback`. The gateway checks the ID token (signature RS256, PS256 or ES256; issuer, audience, expiry, nonce) and maps a claim to a role.
4. The gateway makes a short-lived session token and sends the browser to the admin app with it in the URL fragment (after `##`, which is never sent to a server). The admin app stores it and clears the address bar.

The login attempt is kept in an encrypted, HttpOnly cookie, so the callback may land on any gateway when several share state (doc 22).

21.2 Set it up

Register an application at your provider. Redirect URI: `https://YOUR-CMS/v1/auth/oidc/callback`. Then set:

Setting	Meaning
`SETHUCMS_OIDC_ISSUER`	The provider's issuer address (https, except loopback for tests). Turns SSO on.
`SETHUCMS_PUBLIC_URL`	The address people use to reach the CMS.
`SETHUCMS_OIDC_CLIENT_ID`, `SETHUCMS_OIDC_CLIENT_SECRET`	From the provider.
`SETHUCMS_OIDC_SCOPES`	Default `openid profile email`. `openid` is required.
`SETHUCMS_OIDC_ROLE_CLAIM`	Claim that holds groups. Default `groups`.
`SETHUCMS_OIDC_ADMIN_VALUES`, `_EDITOR_VALUES`, `_VIEWER_VALUES`	Comma-separated group values per role. Highest match wins (admin, editor, viewer).
`SETHUCMS_OIDC_DEFAULT_ROLE`	Role for people who match no group: `editor` or `viewer`. Never `admin`. Leave empty to refuse them.
`SETHUCMS_OIDC_ALLOWED_DOMAINS`	Optional email domains allowed. Needs `email_verified` true.
`SETHUCMS_OIDC_SESSION_HOURS`	Session length. Default 8.
`SETHUCMS_ADMIN_URL`	Where to send the browser afterwards.

Setting	Meaning

21.3 Limits

- Three fixed roles (admin, editor, viewer). Custom roles are not built.
- A person has at most 5 live sessions. `POST /v1/auth/logout` ends one; an admin can revoke any token.
- Group changes at the provider apply at the next sign-in, not to a session already running.
- Sign-ins and denials are in the audit log.

22 Several gateways, and cloud deploys

22.1 Shared state

Set `SETHUCMS_STATE_URL` to a PostgreSQL address and any number of gateways can run against the same state.

What is shared: connections, tokens and sessions, the audit log (one hash chain per gateway process, all verifiable through `/v1/admin/audit/verify`), Idempotency-Key answers, live events (relayed with LISTEN/NOTIFY), the change feed and webhook deliveries (doc 24), and migration job records with a heartbeat so one gateway does not take over a job another is still running.

What is **not** shared:

- **Rate limits are per gateway.** With N gateways the effective limit is up to N times the setting.
- Open database connections stay in the gateway that opened them.
- Token revocation and connection edits reach other gateways within `SETHUCMS_STATE_POLL_MS` (default 1000) or sooner by notification.

Rules in shared mode: secrets must come from the environment (the vault key, cursor secret and preview secret must be the same on every gateway), and the data-folder lock is skipped. Use `SETHUCMS_DATA_DIR=memory` on hosts without a disk.

Status: tested with two gateways on a real PostgreSQL 16 (concurrent writes, revocation, audit chains, idempotency across gateways, event relay). Not tested under real load or with a managed Postgres.

22.2 The worker

Webhooks (doc 24) are sent by a worker. By default each gateway runs one inline; deliveries are claimed with `FOR UPDATE SKIP LOCKED`, so several never send the same one twice at the same time. To run it apart: `SETHUCMS_WORKER=off` on the gateways and `node dist/src/worker.js` (same image, different command) with the same `SETHUCMS_STATE_URL` and secrets.

22.3 Deploy workflows

The files are in `deploy/github-workflows/`. GitHub only reads `.github/workflows/`, so copy them there (they could not be written there from the tool that made them).

File	Does
`sethucms-ci.yml`	Checks out all four repositories, builds, runs the tests with a Postgres service.
`sethucms-release-image.yml`	On a `v*` tag, builds the image and pushes it to GHCR with provenance and an SBOM.
`deploy-cloud-run.yml`	Builds into Artifact Registry and deploys to Cloud Run (Workload Identity Federation, secrets from Secret Manager). Uses `--min-instances 1 --no-cpu-throttling` so the inline worker keeps running.
`deploy-vps-compose.yml`	Copies `compose.production.yml` and the Caddyfile to your server over SSH (pinned host key) and restarts. Secrets stay in `/opt/sethucms/.env` on the server.

Before first use: set the repository variables named at the top of each file, and set `SETHUCMS_OWNER` if the other three repositories are under another owner. Pin the actions to commit SHAs (they use version tags now; the SHAs could not be looked up when they were written).

Status: the YAML parses and passes actionlint. None of the workflows, the image build, the compose file or Caddy have been run. Expect to fix small things on the first run. The image is built for amd64 only.

23 Adapters in other languages (gRPC)

Write an adapter in Python, Go or any language with gRPC, run it as its own service, and add it as a `grpc` connection.

The contract is `sethucms-adapter-sdk/proto/sethucms/adapter/v1/adapter.proto` (protocol version 1). Calls are unary: Describe, Connect, Capabilities, Introspect, Find, Get, Create, Update, Delete, Disconnect, CreateCollection, FinishBulkLoad. Payloads are JSON in string fields, with the same shapes as the TypeScript `Adapter` interface. (`Disconnect` is not named `Close` because gRPC clients already have a `close()`.)

- **Auth:** `authorization: Bearer <token>` metadata. Your service must check it.
- **Errors:** return a gRPC status and put the SethuCMS error code (for example `NOT_FOUND`, `CAPABILITY_UNSUPPORTED`) in trailing metadata `sethucms-error-code`. Messages of internal errors are hidden from API callers.
- **Capabilities:** declare honestly what your database can do. If you declare no inequality filters, the gateway refuses such queries with `CAPABILITY_UNSUPPORTED` instead of sending them.

Connection config: `{ "address": "host:port", "tls": true, "caCert": "-----BEGIN", "callTimeoutMs": 10000 }`, credentials `{ "token": "" }`. In production plain text to a non-loopback host is refused, and the network guard checks the address like any other database.

Write a server in TypeScript with `serveAdapter()` from `@sethucms/adapter-sdk/grpc`. A complete Python server is in `sethucms-adapter-sdk/examples/python-minimal/server.py`. Check yours with the conformance suite (`defineConformanceSuite` in `@sethucms/adapter-sdk/testing`) by pointing a `GrpcAdapter` at it; `apps/api/tests/grpc-python.test.ts` shows how.

Status: tested in-process (TypeScript server) and against the Python example, both passing the conformance suite. TLS with a custom CA is not tested. The older Python and Go template repositories were written before this protocol and probably need updating to match the proto.

24 Change feed and webhooks

Every create, update and delete made **through the gateway** gets a number in an ordered feed.

24.1 Reading the feed

`GET /v1/changes?since=<n>&limit=<k>` returns `{ data, next, head, gap }`. Keep `next` and ask for it next time to resume. `gap: true` means the feed no longer holds changes from your `since`; do a full re-read. Callers only see changes to records they are allowed to see.

24.2 Webhooks (admin only, API only)

- `POST /v1/admin/webhooks { name, url, events }` (`events` is a list of event types, or `["*"]`) returns the signing secret `whsec_` **once**. Also `GET`, `PATCH /:id`, `DELETE /:id`, and `GET /:id/deliveries`.
- Delivery is at least once. Use the `X-Sethu-Delivery` header (`evt_<number>`) to ignore repeats.
- In production the URL must be https. The address is checked by the network guard on every attempt, the connection is pinned to the checked IP, redirects are not followed, 10 s timeout.
- Retries wait 10 s, 30 s, 2 min, 10 min, 30 min, 1 h, 3 h; after 8 attempts the delivery is marked dead.

Verify a delivery (Node):

```
import { createHmac, timingSafeEqual } from 'node:crypto';
function verify(rawBody, header, secret) {
  const parts = Object.fromEntries(header.split(',').map((p) => p.split('=')));
  if (Math.abs(Date.now() / 1000 - Number(parts.t)) > 300) return false;
  const expected = createHmac('sha256', secret).update(`${parts.t}.${rawBody}`).digest('hex');
  const a = Buffer.from(expected), b = Buffer.from(parts.v1 ?? '');
  return a.length === b.length && timingSafeEqual(a, b);
}
```

Header: `X-Sethu-Signature: t=<unix>,v1=<hex>`.

24.3 Not built

- **Native change capture.** MongoDB change streams, Postgres logical decoding and Firestore listeners are not used. Changes made directly in the database, bypassing the gateway, do not appear.
- No admin app screen for webhooks yet.
- Live events (SSE) cannot resume with `Last-Event-ID`; use the feed to catch up.
- The file-based feed keeps up to one second of changes at risk in a crash; the Postgres feed does not.

Status: tested (16 tests plus a standalone worker with two gateways on Postgres 16). Not tested against real receivers on the internet.

25 Use it from your app

SethuCMS is an HTTP API with a TypeScript SDK. Any stack that can make HTTPS requests can use it: Next.js, React, Angular, plain JavaScript, Flutter, Python. There is no framework plugin to install.

25.1 Install the SDK

```
npm install @sethucms/sdk
```

```
import { SethuClient, eq } from '@sethucms/sdk';

const cms = new SethuClient({ baseUrl: 'https://cms.example.com', token: () => getToken() });
const orders = cms.collection('conn_123', 'orders');
const page = await orders.where(eq('status', 'paid')).limit(20).list();
```

The SDK reference is in its README. The `baseUrl` must be https, except for localhost.

The hosted gateway, the viewer token for a website and the safe way to add a content database are in doc 26.

25.2 Where the token lives

The token decides what the caller may read and write (see page 7). Treat it like a password.

Stack	Where to call the SDK	Token
Next.js	Server Components, Server Actions and route handlers	Stays on the server
React or Angular (browser)	In the browser	A short-lived user token from single sign-on (page 21). Never a service token
Plain HTML	In the browser	Same as above; only for local testing with a test token

25.3 Example apps

The `sethucms-examples` repository has the same small app in four stacks: plain HTML, React (Vite), Angular and Next.js. Each lists your connections and collections, shows and edits records, follows live changes, and runs a self-check against your gateway. The React and Angular examples keep the token in the browser, which is fine for local testing only. The Next.js example keeps it on the server and is the pattern to copy for production.

25.4 From install to a live site

Six steps tie the pages together. (1) Run a gateway; scheduled publishing needs one that keeps running, so not serverless mode. (2) Connect the database read-only and expose only the collections the site reads; an optional schema file sets labels and rules and marks a collection `singleton: true`. (3) In the website's server code, create a `SethuClient` with a viewer token kept in the server environment, then call `where(...).list()`, `query().first()` for a singleton, and `count()`; cache the page so the gateway is asked about once a minute. (4) Build the content-only editor into `public/sethu`, add the website's exact address under *CORS origins* (or use *Allow this site* after the first refused call), and give each editor an Editor account. (5) Editors preview with a signed link that needs no token, and schedule from the editor or with `drafts.schedule`; the gateway publishes at the time. (6) A page the website has cached shows the change when its cache expires. The common failures are a missing CORS entry ("blocked by CORS policy"), a token with too small a role (`403`), and scheduling on a serverless gateway (`CAPABILITY_UNSUPPORTED`). Not yet tested: these steps on a live customer site, and counts and scheduling on MySQL, MongoDB and Firestore.

25.5 What is not there yet

- No `create-sethucms` starter and no React hooks or Angular service library. You use the SDK directly.
- Nuxt and other frameworks should work the same way, but they have not been tested.
- Row-level rules (for example, a vendor sees only their own rows) are supported by the permission system, but a full multi-vendor app has not been tested end to end.
- No vendor or business dashboards yet.

26 Hosted gateway, admin app and your first content connection

This page covers the first live setup: the gateway on Vercel, the admin web app that comes with it, and how a website reads its content from a database it owns. It is written from the real 0.1.0 launch, including the mistakes worth avoiding.

26.1 What is running

Piece	Where	Notes
Gateway and admin web app	Vercel, one origin	The admin app is served by the gateway. It uses a hash router and keeps the sign-in token in `sessionStorage`, so closing the tab signs you out
Gateway state	A Firestore project, collection prefix `sethucms`	Connections, tokens, sessions, the audit log, idempotency answers. Serverless functions share no disk, so state must live in a database (doc 22)
Your content	Your own database, added as a **connection**	Kept separate from the state project
Packages	npm, scope `@sethucms`, version 0.1.0	Ten packages: core, adapter-sdk, sql-kit, postgres, mysql, sqlite, mongodb, firestore, memory, sdk-ts

Keep two Firebase projects: one for the gateway's own state and one for your content. If they are the same project, the state collections (connections, tokens, audit) sit next to your content and are one mistake away from being exposed.

26.2 Sign in to the admin app

1. Open the gateway address in a browser.
2. Paste the admin token. It is an **admin** token created when the gateway first started; it is not a viewer token.
3. Create a separate viewer token for each website (Access, then New token). Give it the least role it needs.

Treat the admin token like a root password: never paste it into a chat, a ticket or a screenshot, and clear your terminal after printing it.

26.3 Add your content database

In the admin app go to Connections, then Add, then choose Firestore. You need:

- `projectId`: the Firebase project that holds your content.
- `databaseId`: leave as `(default)` unless you made a named database.
- A **service-account key** (JSON) for that project. Download it from the Firebase console, store it as a file with mode 600, and paste it into the credentials box. The vault encrypts it; it is never shown again.

Then set:

- **Mode: read only.** A website that only shows content never needs to write.
- **Expose: none.** Start with every collection hidden, then expose only the ones the site reads, for example `projects`.

Check the result with the connection detail page. It should list only the collections you chose as exposed, and the health check should be green.

26.3.1 The mistake to avoid

During the launch the first connection was created with expose set to all, and it was pointed at the state project. That published twelve internal collections (connections, tokens, audit and others) to every viewer token. The connection was deleted, every token revoked and new ones issued. The rules that came out of it:

- Never use `expose: all` on a project that also holds gateway state.
- Create connections with `expose: none` and add collections one at a time.
- After any change, list what a viewer token can see (`GET /v1/connections/:id/collections`) and confirm it is only what you intended.

26.4 Read the content from a website

Install the SDK in the website, never in browser code that a visitor can inspect.

```
npm install @sethucms/sdk
```

Put three values in the website's server environment:

Variable	Value
`SETHUCMS_BASE_URL`	The gateway address (https)
`SETHUCMS_TOKEN`	A **viewer** token made for this site
`SETHUCMS_CONNECTION_ID`	The id of the content connection (it starts with `conn_`)

```
import { SethuClient, eq } from '@sethucms/sdk';

const cms = new SethuClient({
  baseUrl: process.env.SETHUCMS_BASE_URL!,
  token: () => process.env.SETHUCMS_TOKEN!,
});

export async function listProjects() {
  return cms
    .collection(process.env.SETHUCMS_CONNECTION_ID!, 'projects')
    .where(eq('published', true))
    .limit(50)
    .list();
}
```

In Next.js, call this from a Server Component or a route handler and cache the page (for example `revalidate: 60`) so the gateway is asked at most once a minute, not once per visitor. The token stays on the server.

26.5 The state store in serverless mode

Several functions can handle requests at the same moment, so the shared state store updates the state document with a revision check: read the revision, write the change only if the revision is unchanged, otherwise wait a short random time and try again, up to twenty times. If all twenty attempts lose, the request fails with a clear error instead of overwriting someone else's change.

The audit log in this mode writes one document per entry. Each entry carries an ascending **newest-first key** (a padded timestamp counter subtracted from a large constant), so "latest entries" is an ascending scan with a limit. This is done because the Firestore emulator, and some Firestore setups, do not support descending key scans.

Status: seven live tests pass against the Firestore emulator (concurrent updates, revision conflicts, audit order across two gateways). The Firestore content adapter itself is still lightly tested; run it against a test project before relying on it (doc 18).

26.6 Operating notes

- Redeploy after changing code: build the Vercel bundle, then deploy it to production. Environment variables are read at deploy time.
- Turn off Vercel Deployment Protection for the gateway project, otherwise browsers and the SDK get a login page instead of JSON.
- Rate limits are per function instance, so the real limit is higher than the configured one (doc 22).
- Rotate any secret that has been pasted into a chat, a screenshot or a terminal log. Revoke one-day publishing tokens as soon as the release is done.

27 Roadmap

Where we are (October 2026)

Phase	State
Foundation	Done: core (29 tests), adapter contract and test kit
Security spine	Done: vault, token auth, audit log, network guard, rate limits, idempotency, single sign-on (tested only against a fake identity provider)
First path	Done: PostgreSQL adapter and gateway, tested against a live database
More adapters	Done: memory, PostgreSQL, SQLite, MySQL/MariaDB (real servers), MongoDB and Firestore (stand-ins only). Not started: SQL Server, DynamoDB
Admin UI	Done for the first release: connections, content editing with drafts, publish, live preview, access, audit, settings
SDKs	TypeScript SDK published as 0.1.0 and used by four example apps and a live website. Python and Dart written but never run
Hardening	Not started: load tests, penetration review, key-rotation drill, cloud-identity connections

Also done beyond the original plan: CI/CD workflows for every repository, a one-image release

build, agent guides, all ten 0.1.0 packages on npm, and a hosted gateway with the admin app on Vercel with Firestore state (seven live state-store tests pass against the Firestore emulator). Next, in order: run the container image, Compose file and deploy workflows end to end; run the MongoDB and Firestore adapter tests against real servers; native change capture from the databases; custom roles and webhook screens in the admin app; business and vendor dashboards; a `create-sethucms` starter and helper libraries for React and Angular.

Also done in November 2026: record counts, singleton collections, scheduled publishing, the CORS origins screen with one-click allow, a brand file (`theme.css`) with contrast fixes in the admin app, and `@sethucms/sdk` 0.2.0. These were tested against the memory adapter and, for the multi-gateway publish claim, one PostgreSQL 16 server. Not yet checked: MySQL, MongoDB and Firestore with these features, a screen reader, and real phones.

Estimates assume one or two engineers and are rough planning guesses, not benchmarks.

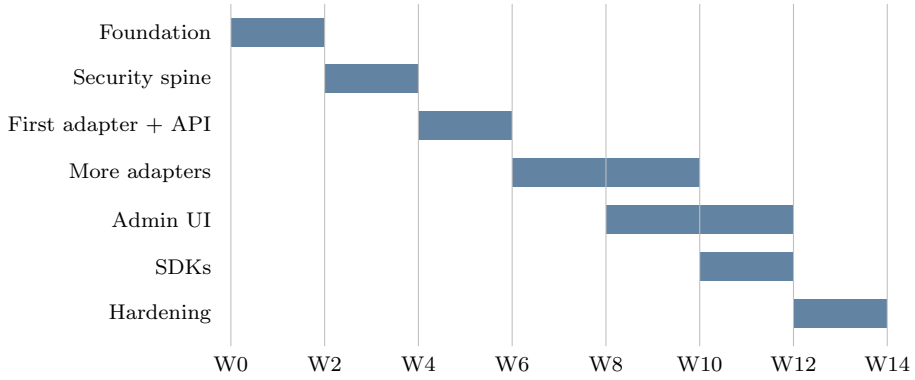


Figure 6: MVP plan in weeks.

Phase	Goal	Exit criteria
1. Foundation	Repo, core types, adapter protocol, control-plane schema	core builds; migrations apply cleanly
2. Security spine	Vault, auth, audit, network guard	Secrets encrypted end to end; SSRF tests pass
3. First path	Postgres adapter plus API	Conformance suite green on Postgres
4. More adapters	MongoDB, Firestore, MySQL, SQL Server, DynamoDB	Each passes conformance and security suites
5. Admin UI	Connect, introspect, edit, permissions	An editor completes a full content edit through the UI only
6. SDKs	Generated Dart and Python, hand-tuned TypeScript	Sample apps (Next.js, Flutter, FastAPI) run
7. Hardening	Load, security review, docs, cloud identity	Security review and key-rotation drill done

27.1 Hand-written files for the MVP (estimate)

Area	Files	Area	Files
Root config	12	api	25
core	15	worker	12
protocol + adapter-sdk	13	admin	40
6 adapters	36	control-plane DB	10
vault	8	sdk-ts	8
auth	10	Dart SDK	8
audit + network-guard	11	Python SDK	9
infra	15	adapter templates (Go, Python)	16
tests	20	CI + docs	20
Total (approximate)			**288**

27.2 Risks

Risk	Mitigation
Capability gaps make some databases feel second class	Honest capability flags; clear UI messaging
Credential handling mistakes	Security spine before any adapter; key-rotation drill
Scope creep across databases	Ship six adapters; add others on demand
Driver or ORM version changes (for example Prisma and MongoDB)	No single ORM for all stores; isolate drivers inside adapters
Licensing of embedded tools	Prefer MIT/Apache components; legal review before embedding source-available tools

A Adapter Protocol (proto)

```
// SethuCMS adapter protocol.
// An adapter implements this service in any language. The gateway calls it.
syntax = "proto3";

package sethucms.adapter.v1;

import "google/protobuf/struct.proto";

service Adapter {
  rpc Connect(ConnectRequest) returns (ConnectResponse);
  rpc Introspect(IntrospectRequest) returns (IntrospectResponse);
  rpc Capabilities(CapabilitiesRequest) returns (CapabilitiesResponse);
  rpc Find(FindRequest) returns (FindResponse);
  rpc Get(GetRequest) returns (GetResponse);
  rpc Create(WriteRequest) returns (WriteResponse);
  rpc Update(WriteRequest) returns (WriteResponse);
  rpc Delete(DeleteRequest) returns (DeleteResponse);
  rpc Batch(BatchRequest) returns (BatchResponse);
  rpc Subscribe(FindRequest) returns (stream ChangeEvent);   // optional
  rpc Close(CloseRequest) returns (CloseResponse);
```

```
}

// Credentials are resolved by the worker and passed in memory only; never logged.
message ConnectRequest {
  string connection_id = 1;
  string adapter_type = 2;
  google.protobuf.Struct config = 3;
  google.protobuf.Struct credentials = 4;
  bool read_only = 5;
}
message ConnectResponse { bool ok = 1; Error error = 2; }

message IntrospectRequest { string connection_id = 1; }
message IntrospectResponse {
  repeated CollectionInfo collections = 1;
  Error error = 2;
}
message CollectionInfo {
  string native_name = 1;
  repeated FieldInfo fields = 2;
  repeated RelationInfo relations = 3;
}
message FieldInfo {
  string native_name = 1;
  string native_type = 2;
  string canonical_type = 3;
  bool required = 4;
  bool is_primary_key = 5;
  bool lossy = 6;
}
message RelationInfo {
  string name = 1;
  string target = 2;
  string kind = 3;
  google.protobuf.Struct mapping = 4;
}

message CapabilitiesRequest { string connection_id = 1; }
message CapabilitiesResponse {
  bool joins = 1;
  string transactions = 2;      // none | single-document | multi-document
  bool filter_or = 3;
  string filter_inequality = 4; // full | single-field | none
  bool full_text = 5;
  string sort = 6;              // any-field | indexed-only | key-only
  bool offset_pagination = 7;   // cursor pagination is mandatory
  bool aggregations = 8;
  bool realtime = 9;
  string schema = 10;           // enforced | inferred
  bool writes = 11;
}

message FindRequest {
  string connection_id = 1;
  google.protobuf.Struct query = 2;   // portable query AST (see query-ast.schema.json)
}
message FindResponse {
  repeated google.protobuf.Struct records = 1;
  string next_cursor = 2;
  repeated string capability_notes = 3;
  Error error = 4;
}

message GetRequest { string connection_id = 1; string collection = 2; string id = 3; }
```

```
message GetResponse { google.protobuf.Struct record = 1; Error error = 2; }

message WriteRequest {
  string connection_id = 1;
  string collection = 2;
  string id = 3;                        // empty on create
  google.protobuf.Struct values = 4;
}
message WriteResponse { google.protobuf.Struct record = 1; Error error = 2; }

message DeleteRequest { string connection_id = 1; string collection = 2; string id = 3; }
message DeleteResponse { bool ok = 1; Error error = 2; }

message BatchRequest { string connection_id = 1; repeated WriteRequest writes = 2; bool atomic =
    3; }
message BatchResponse { repeated WriteResponse results = 1; Error error = 2; }

message ChangeEvent {
  string type = 1;                      // insert | update | delete
  google.protobuf.Struct record = 2;
}

message CloseRequest { string connection_id = 1; }
message CloseResponse { bool ok = 1; }

message Error {
  string code = 1;  // CONNECTION_FAILED, AUTH_FAILED, NOT_FOUND, CONFLICT, VALIDATION_FAILED,
                    // CAPABILITY_UNSUPPORTED, RATE_LIMITED, TIMEOUT, READ_ONLY, INTERNAL
  string message = 2;
  google.protobuf.Struct details = 3;
}
```

B Architecture Decision Records

ADR 0001: Language choice

Proposed, 2026-10-08

Decision: TypeScript for core, adapter-sdk, first-party adapters, api, worker and admin; generated Dart and Python SDKs; adapters in any language through a language-neutral protocol. **Revisit when** worker CPU or connection counts become the bottleneck, or a customer needs an adapter whose best driver is not on Node.

ADR 0002: Adapters as processes behind a protocol

Proposed, 2026-10-08

Decision: define the adapter contract in `adapter.proto` and run adapters as separate worker processes, with an in-process fast path for trusted first-party adapters. **Gains:** any language, per-connection isolation, one conformance kit. **Costs:** a network hop and more services to operate.

ADR 0003: CMS metadata lives in the control plane

Proposed, 2026-10-08

Decision: store all CMS metadata in a separate control-plane Postgres; customer databases hold only customer data; connections are read-only by default. **Cost:** schema drift must be caught by re-introspection and reconciled with stored metadata.